

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Report

Laboratory work No. 3

The Command Line: Shell, Pipes and Text Filters

Done by: st. gr. FAF-241, Coșneanu Corina

Checked by:
Reițman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chișinău – 2026

Contents

ALGORITHM ANALYSIS AND OBJECTIVES 1

1 Part 1: Move fast, find help 2

2 Part 2: What the shell does to your line 3

3 Part 3: Redirection, chaining and jobs 5

4 Part 4: Answer questions with one-liners 8

5 Part 5: Your first script 10

CONCLUSIONS 11

ALGORITHM ANALYSIS AND OBJECTIVES

Objective

The objective of this laboratory work is to master the Linux command-line shell, understand how the shell processes command lines prior to execution, manage standard input/output streams through redirection and pipes, and build efficient text-processing pipelines using standard Unix filters.

Tasks

1. Study basic shell navigation, use autocompletion (`Tab`), command history, and differentiate between shell built-ins and external binaries.
2. Analyze pre-execution shell mechanisms including wildcard expansion (globbing), quote handling, and variable interpolation.
3. Practice redirection of standard streams, chaining operators, and job control in the background.
4. Solve analytical questions using single-line pipelines (`grep`, `cut`, `sort`, `uniq`, `wc`, `tail`) on an artificial web access log (`access.log`).
5. Implement a robust Bash script that accepts a log file as an argument, validates its existence, and generates statistical failure summaries for individual users.

Theoretical Notes

A shell acts as a powerful command interpreter linking the user to the kernel. A key concept explored in this work is the **pipeline** (represented by `|`), which routes the standard output of one utility directly into the standard input of another without requiring intermediate temporary files.

The principal text-processing utilities utilized comprise:

- `grep` – searches text files for matching regular expression patterns;
- `cut` – extracts designated fields or columns from text lines;
- `sort` – sorts lines of text alphabetically or numerically;
- `uniq` – filters or counts adjacent duplicate lines;
- `wc` – counts lines, words, and characters in data streams;
- `tail` – outputs the final lines of a target file.

1 Part 1: Move fast, find help

This part explores command identification and system documentation.

- **Built-ins vs Externals:** `cd` and `echo` are built directly into the shell, whereas `ls` is an external executable file. `cd` must be a built-in because a child process cannot modify the working directory of its parent shell process.
- **PATH and Exit Codes:** The system scans directories specified in the `$PATH` variable. An invalid command returns the standard error code **127**.
- **Sorting by modification time:** The option `-t` in `ls` sorts files by modification time, newest first.



Figure 1.1: Tab completion

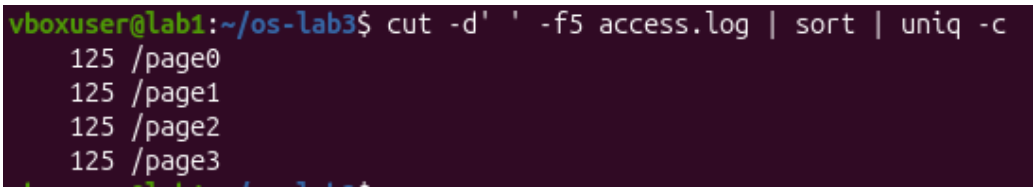


Figure 1.2: Output of type and which

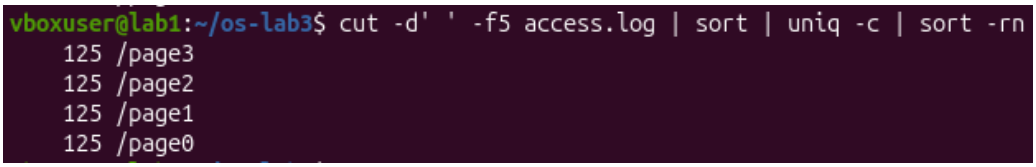


Figure 1.3: The directories in PATH

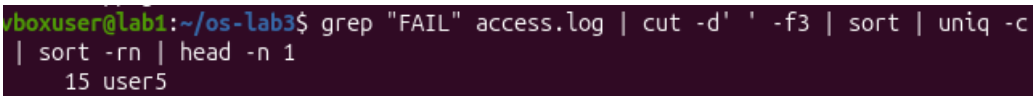


Figure 1.4: Exit code of a missing command

2 Part 2: What the shell does to your line

Before executing commands, the shell interprets special characters, performs wildcard expansion (globbing), and strips quotes.

Command	My Prediction	Real Output	Explanation (if different)
`echo *.c`	a.c b.c	a.c b.c	Shell expands `*.c` into matching filenames.
`echo file?.txt`	file1.txt file2.txt	file1.txt file2.txt	'?' matches any single character.
`ls alfa beta.txt`	Error / Not found	ls: cannot access...	Spaces split arguments into multiple targets.
`ls "alfa beta.txt"`	alfa beta.txt	alfa beta.txt	Quotes preserve spaces inside the filename.

Table 1: Globbing and quoting predictions table

Differences between quotes: Single quotes (`' '`) preserve the literal value of every character within them (no variable expansion). Double quotes (`" "`) preserve the literal value of most characters but allow the shell to expand variables (like `$HOME`) before passing the argument to the command.

Why ls shows files with quotes: When a filename contains a space, `ls` may output it surrounded by single quotes (e.g., `'alfa beta.txt'`) so that the user clearly sees it as a single filename entity rather than two separate files.

```
vboxuser@lab1:~/os-lab3$ echo "Total requests by user3: $(grep -c 'user3' access
.log) | Failed: $(grep 'user3' access.log | grep -c 'FAIL')"
Total requests by user3: 72 | Failed: 15
```

Figure 2.1: Files created in the play directory

```
vboxuser@lab1:~/os-lab3$ grep "FAIL" access.log | tail -n 3 | cut -d' ' -f2,3
10:10 user0
10:15 user5
10:20 user3
```

Figure 2.2: Globbing results

```
vboxuser@lab1:~/os-lab3$ cut -d':' -f7 /etc/passwd | sort | uniq -c | sort -rr
     40 /usr/sbin/nologin
      5 /bin/false
      2 /bin/bash
      1 /bin/sync
```

Figure 2.3: The ls error on a name with a space, and the fix

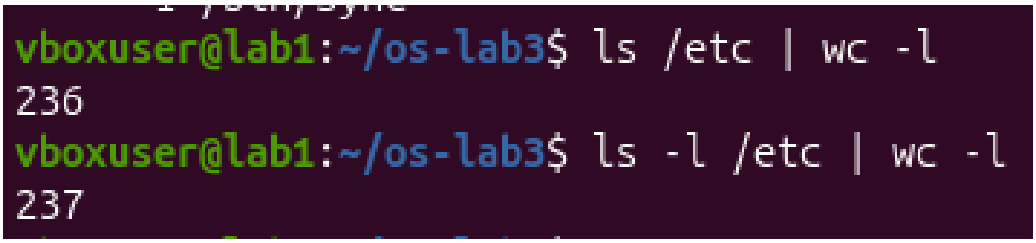


Figure 2.4: Single quotes, double quotes and backslash

3 Part 3: Redirection, chaining and jobs

Stream redirection allows splitting standard output (`>`) from standard error (`2>`).

- **File distribution:** When running `ls /etc/nope > out.txt 2> err.txt`, the error message goes to `err.txt` and the standard listing (which is empty in this case) goes to `out.txt`.
- **Redirection order:** The command `> all.txt 2>&1` accurately merges both streams into a single target file because stderr is redirected to where stdout currently points (the file). Conversely, `2>&1 > all.txt` redirects stderr to the screen (where stdout initially pointed) and only then redirects stdout to the file.
- **wc and filenames:** `wc -l < access.log` prints only a number because the shell feeds the file content directly into standard input, so `wc` doesn't know the filename. `wc -l access.log` passes the file as an argument, so `wc` reads it and prints the name alongside the count.
- **Logical operators:** `;` executes commands sequentially regardless of success; `&&` executes the second command only if the first succeeds (exit code 0); `||` executes the second command only if the first fails.

```
vboxuser@lab1:~/os-lab3$ bash oneliners.sh
100
    125 /page0
    125 /page1
    125 /page2
    125 /page3
    125 /page3
    125 /page2
    125 /page1
    125 /page0
     15 user5
Total requests by user3: 72 | Failed: 15
10:10 user0
10:15 user5
10:20 user3
     40 /usr/sbin/nologin
      5 /bin/false
      2 /bin/bash
      1 /bin/sync
Bonus - Failed at exact 10 minutes:
50
```

Figure 3.1: stdout and stderr in separate files

```
vboxuser@lab1:~/os-lab3$ cat report.sh
#!/bin/bash
#usage: ./report.sh access.log
log=$1

if [ ! -f "$log" ]; then
    echo "no such file: $log" >&2
    exit 1
fi

total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")
echo "$total requests, $fails failed"

#TODO: for each of user0 ... user6, print "userN: X failures"
for i in {0..6}; do
    count=$(grep "user$i" "$log" | grep -c FAIL)
    echo "user$i: $count failures"
done
```

Figure 3.2: Comparing the order of redirections

```
vboxuser@lab1:~/os-lab3$ ./report.sh access.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
```

Figure 3.3: The ; && || operators

```
vboxuser@lab1:~/os-lab3$ ./report.sh nope.log; echo "exit code: $?"
no such file: nope.log
exit code: 1
```

Figure 3.4: Running a background job and stopping it

4 Part 4: Answer questions with one-liners

Question 1: Number of Failed Requests

```
grep -c "FAIL" access.log
```

Figure 4.1: Number of failed requests

Question 2: Different Requested Pages

```
cut -d' ' -f5 access.log | sort | uniq -c
```

Figure 4.2: Different requested pages

Question 3: Number of Requests for Each Page

```
cut -d' ' -f5 access.log | sort | uniq -c | sort -rn
```

Figure 4.3: Number of requests for each page

Question 4: Users with the Most Failed Requests

```
grep "FAIL" access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn | head -n 1
```

Figure 4.4: Users with the most failed requests

Question 5: Requests Made by user3

```
echo "Total requests by user3: $(grep -c 'user3' access.log) | Failed: $(grep '
    user3' access.log | grep -c 'FAIL')"
```

Figure 4.5: Requests made by user3

Question 6: Last Three Failed Requests

```
grep "FAIL" access.log | tail -n 3 | cut -d' ' -f2,3
```

Figure 4.6: Last three failed requests

Question 7: Login Shells from /etc/passwd

```
cut -d':' -f7 /etc/passwd | sort | uniq -c | sort -rn
```

Figure 4.7: Login shells from /etc/passwd

Question 8: Comparing Line Counts

```
ls /etc | wc -l
ls -l /etc | wc -l
```

Explanation: The outputs differ by exactly one because `ls -l` includes a summary header line ('total ...') at the top of the listing.

Figure 4.8: Comparing Line Counts

Bonus: Failed at exact 10 minutes

```
echo "Bonus - Failed at exact 10 minutes:"
grep -E "10:[0-5]0.*FAIL" access.log | wc -l
```

Figure 4.9: Full run of oneliners.sh including bonus

5 Part 5: Your first script

A script named `report.sh` was implemented to automate log file verification, argument handling, error emission to standard error (`>&2`), and iterative per-user failure computation.

Execution before chmod +x: The `#!/bin/bash` line is a shebang that tells the system to execute the file using the Bash interpreter. If the script is executed via `./report.sh` before running `chmod +x`, the shell returns a "Permission denied" error because the file lacks executable permissions.

Execution requirements: You must type `./report.sh` instead of just `report.sh` because the current directory (`.`) is not included in the system's `$PATH` variable for security reasons.

Error handling: The script uses `>&2` to redirect the error message to standard error (so it isn't accidentally processed as standard data) and uses `exit 1` to signal to the calling shell that the script terminated unsuccessfully.

```
#!/bin/bash
#usage: ./report.sh access.log
log=$1

if [ ! -f "$log" ]; then
    echo "no such file: $log" >&2
    exit 1
fi

total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")
echo "$total requests, $fails failed"

#TODO: for each of user0 ... user6, print "userN: X failures"
for i in {0..6}; do
    count=$(grep "user$i" "$log" | grep -c FAIL)
    echo "user$i: $count failures"
done
```

Figure 5.1: Contents of report.sh

Figure 5.2: Running the script on access.log

Figure 5.3: Error handling for a missing file

CONCLUSIONS

This laboratory work provided profound insight into the core mechanics of the Linux command-line interface. By investigating pre-execution shell expansions, globbing patterns, and quoting rules, I enhanced my ability to write predictable shell commands.

Furthermore, mastering stream redirection and command pipelines demonstrated that complex data aggregation tasks can be accomplished cleanly without developing heavyweight standalone software. The utilization of standard filters such as `grep`, `cut`, `sort`, and `uniq` proved essential for efficient log analysis. Finally, encapsulating these logical pipelines inside a structured Bash script with explicit parameter validation and proper error handling reinforced robust programming principles for system administration and task automation.